

QQ 飞车 Agentic 研发转型过程中的 Loop Engineering

演讲人：任磊达

腾讯 / 高级后台开发工程师、项目组 Agent 落地负责人

Loop Engineering：把 Agent 织进正在运转的循环，从而释放人的注意力。

AiCon

全球人工智能开发与应用大会

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

InfoQ 技术大会

QCon

上海站

全球软件开发大会

大模型正在重新定义软件

Large Language Model Is Redefining
The Software

10月22-24日 / 上海建工浦江皇冠假日酒店

· AI Agent

· Vibe Coding

· 智能可观测

· 推理基建

· 模型攻防

· AI x 创造力

InfoQ 技术大会

AiCon

北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

12月18-19日 / 北京

· 大模型架构创新

· 多模态 AI 产业融合

· 具身智能

· AI for Science

· 大模型安全



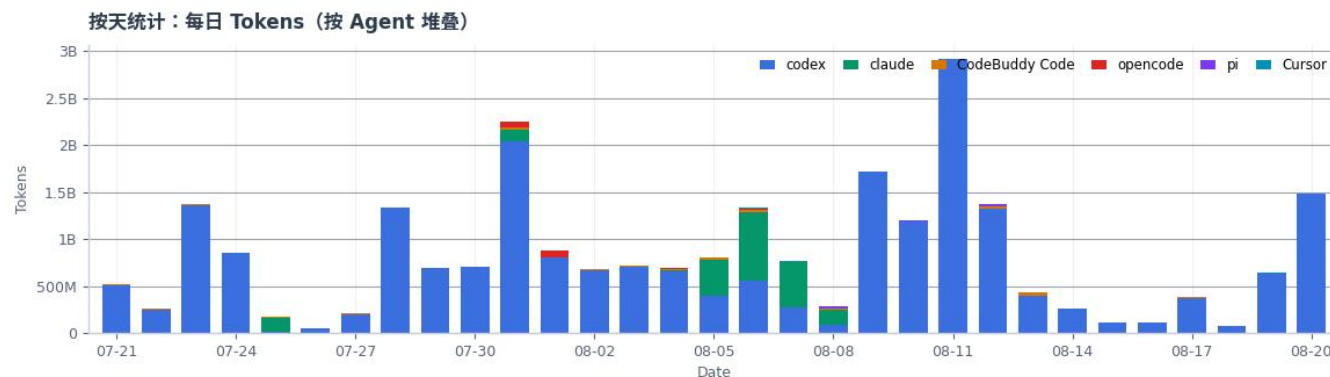
专题演讲嘉宾：任磊达

腾讯 QQ 飞车项目组

高级后台开发工程师、项目组 Agentic 研发转型负责人

本次分享聚焦百人游戏研发团队 Agentic 实践和思考。

Token 已经是生产资料 (07-21 → 08-20)



注：按天统计含 Cursor 最新上下文占用；时段为 Asia/Shanghai

Token 作为生产资料, Loop 是在浪费 Token?

主张

Harness 创建可能性, **Loop** 锚定价值

Token Maxxing: 尽可能升级 human in / on / out the loop.

把 token 花在让 loop 更完整上, 而不是更贵的单次问答上。

质疑

不花 token, 这些工作我也能做

- 文档 / Spec 和实现永远对不上
- OpenClaw、Hermes、dsh 工具层出不穷
- model 慢慢 loop 进 agent 能力, 我们能 loop 什么

回应

Token ROI: 什么时候 in / on / out

human in the loop, human build the loop.

人的注意力只投在判断上, 执行交给 loop。

开场三问 · 结尾回收

① 什么时候 in / on / out the loop?
(Token ROI)

② Loop 多花几倍 token, 是否也有心流价值?

③ 超级个体是否还有价值?

“I Don't Prompt Claude Anymore, My Job Is to Write Loops.” — Boris Cherny

把 idea 构建到 loop，同类问题只犯一次

直接提示 (Prompting Agent)

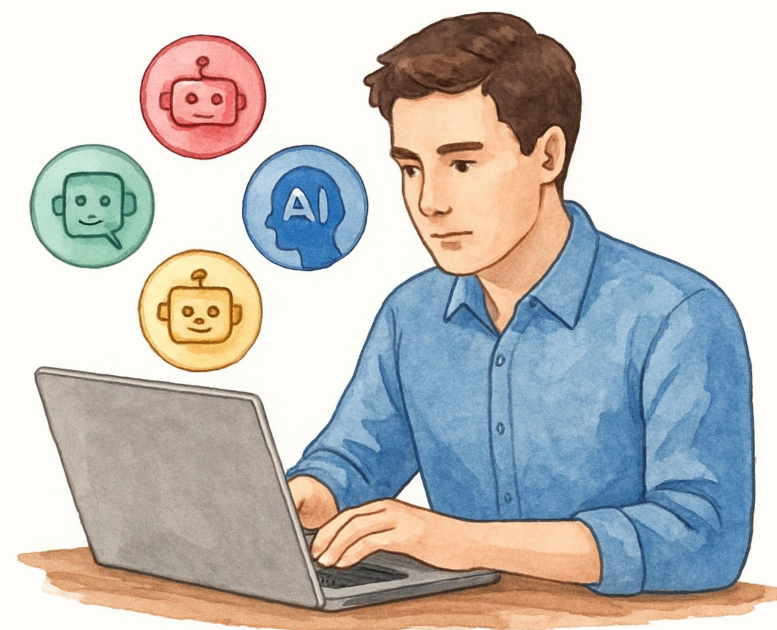
“这段代码太长了，帮我精简一点。”



实现 task

编写循环 (Writing Loops)

“你得主动询问并修复，这段代码是否冗长或有优化空间？”



构建 task (规划、编排、委派)

4 / 51

Loop Engineering: 吴恩达的三层，工程里的四层

维度	吴恩达三层 (Agent 视角)	本分享四层 (工程视角)
循环的宿主	Agent 内部的推理 - 行动 - 观察	工程里本来就在转的 custom-linter / hook / CI / work 循环；工程循环之外，还有多人协作的 Graph
分层的维度	单 Agent → 任务编排 → 多 Agent 工作流	秒 hook → 分钟 CI → 小时/天 work → 组织 Graph
回答的问题	怎么把 Agent 系统编排得更强？	前三层：把 Agent 织进已有循环，而不破坏它 第四层：个人 loop 如何变成团队产能？
落地路径	从零设计 Agent 架构	增量接入：先挂 hook，再接 CI，再嵌 work loop，再连 Graph

吴恩达三层：一圈比一圈大



一层管自己怎么转，一层管任务怎么拆，一层管多个 Agent 怎么协作——每一层的反馈来源都不一样。前三层焊位置，第四层连网络。

Loop 是迭代：先明确迭代的方向

三个迭代方向

① 增加一行 loop 把 skill 删掉 20% 的 token

模型迭代越来越快，skill 删了后

② 对接 LLM，迭代 Agent

LLM 成本随 token 用量持续增长，调用越重、账单越贵

③ 构建 Skill Evaluation

Agent x model 组合上百了，模型迭代越来越快

两个思考

① 这些是 exercise，还是 loop？

② SDD 是 Context Engineering 的 best practice——那 Loop Engineering 的 best practice 是什么？

案例 | FARS: workflow 治理下的全自动科研

四段 workflow 治理



≈ 1,000 美元

一篇论文的平均成本

137 分钟

一篇论文的平均耗时

417 小时

连续运行，无人值守

166 篇

产出的论文总数

workflow 治理到位，loop 才能连续几百小时稳定地产出。

所以，接下来的内容是 workflow 本身

我们如何打磨自己的 workflow：在提高并发度的情况下，确保稳定性。



分享整体结构

开场 | Loop 的本质 = 注意力分配

第一章 | Loop Engineering

① 本地 hook loop

秒级 · 人不出现

② CI loop

分钟级 · review 点出现

③ work loop

小时/天级 · 守边界标准

④ graph loop

组织级 · 连成一张图

第二章 | Engineering Loop

故事一

200k 装不下

故事二

递归雪崩

故事三

4.4 万行

尾声 | RSI: 优化 loop 的 loop × 生活

故事旁色点 = 该坑被固化回哪一层

第一层

本地 hook: 即时、自动地 提供给 Agent 上下文

Agent 写文件 · 秒级

Git commit · 十秒一分钟级

Agent 写代码时的确定性错误, 能不能在产生的那一秒就被消化?

custom-linter 是怎么炼成的：用 AI 开发约束 AI 的工具

启示来自海外大厂：内部最耗精力的不是写功能，而是和 linter 斗争——所以最高 ROI 的场景是 lint → fix → commit。

目标日志格式：六条铁律

```
log.Infof("@ Event=xxx UID=%d Ret=%d Error='%s', ...)
```

- ① 固定 @ 开头，后空一格
- ② 每段是 驼峰Key=值
- ③ UID/Ret 只认关键字，不要 uid/uin/iRet
- ④ 不打 File/Line，交给 zaplogger
- ⑤ 含空格的字段加单引号
- ⑥ 不允许连续空格

为什么用 linter，不用 rules

rules: 求模型别犯错
概率模型，规则越多遵守率越低



linter: 先检测、再修复
像 TDD: 先 Red 再 Green, 永久生效

1 天做完: golangci-lint module plugin

上午调研
plugin 机制



人写需求
custom-linter.md



AI 写代码
+ 单测 testdata



custom-gcl
编进二进制

```
# .custom-gcl.yml → 把 logmessage 编进二进制
plugins:
  - module: 'herokingdom/linter/logmessage'
# .golangci.yml → enable + 白名单 [Event, UID, Ret, Error]
```

报错就是活文档：看到就懂怎么改

递归点：人定规则，AI 写 linter，linter 反过来约束 AI。

lib/wecom/demo/main.go:395:2: segment "签名生成:" is not in Key=Value format (logmessage)

8 / 51

一次 lint 错误的一生: PostToolUse Hook 全程追踪

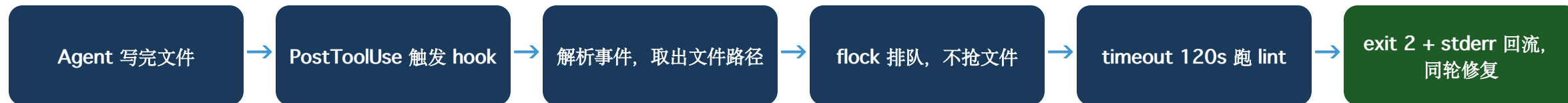
POSTTOOLUSE HOOK



就近原则——问题在产生的那一秒被消化, 不积压到 commit。

就近原则 · PostToolUse Hook: 写完立刻检查, 问题当场回流

一次保存的完整回路 (秒级, 人不在场)



坑 ①: 换一家 CLI, hook 就得重写?

Claude Code、CodeBuddy 的事件是 `tool_input.file_path`, Codex 是 `apply_patch`, 格式不一样。

对策: 一份脚本两种格式都认, 换 CLI 不用改。

```
tool_input.file_path | patch.path
```

坑 ②: 多 Agent 并发, lint 互相打架

多个 hook 实例同时触发, 对同一文件并发跑 lint 会产生竞态。

对策: flock 排他锁, 同一时刻只跑一个 lint。

```
flock -x 9
```

坑 ③: hook 被强杀, lint 变僵尸

CLI 的 hook 超时默认 120s: hook 到点被杀, lint 进程却还在跑。

对策: timeout 与 120s 对齐, 超时自己先退出。

```
timeout 120 golangci-lint run
```

hook 三原则: 兼容、排队、有超时——反馈在同一轮回到 Agent, 人不用出现。

10 / 51

pre-commit Hook: 三个工程取舍

Agent 内 hook 会因为超时、并发冲突和上下文漂移而失效；Git commit 前的 pre-commit 是第二道确定性护栏。

取舍一：只扫暂存文件所在包

60s 单 job 预算逼出增量设计。不扫全仓，只扫本次变更涉及的包，把检查范围压到最小。

原则：检查范围跟着变更走

取舍二：自动修复后必须复验

custom-linter 封装格式修复与规则检查。自动修复、重新暂存、再跑一遍，确保修复结果本身也通过。

原则：修复不是终点，复验才是

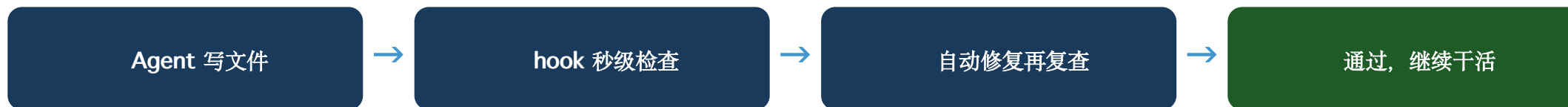
取舍三：Agent 会习惯性用 git commit -n 绕过检查

对策：AGENTS.md 明令禁止 -n，配合 lintignore 白名单留痕，绕过行为可见、可审计

原则：护栏本身也要被监控

第一层 · 注意力答卷：本地 hook loop

秒级循环 · 人几乎不出现



人只看什么

- CI 红灯时的 review hook 配置
- 新增 hook 规则 (work loop 回灌的决策)

Agent 自转什么

- 写完即检: exit 2 + stderr, 同轮修复
- pre-commit: 多 job 并发, 只扫暂存文件所在包
- ruff --fix 后自动 re-add 再复查



分享整体结构

开场 | Loop 的本质 = 注意力分配

第一章 | Loop Engineering

① 本地 hook loop
秒级 · 人不出现

② CI loop
分钟级 · review 点出现

③ work loop
小时/天级 · 守边界标准

④ graph loop
组织级 · 连成一张图

第二章 | Engineering Loop

故事一
200k 装不下

故事二
递归雪崩

故事三
4.4 万行

尾声 | RSI: 优化 loop 的 loop × 生活

故事旁色点 = 该坑被固化回哪一层

📄 第一层收口: lint 错误活不过一次工具调用, 人不出现。

代码离开本地之后, 合入这个循环谁来守?

9.5% 的 MR 能走 auto-approve 快线, 平均 8.5 分钟合入

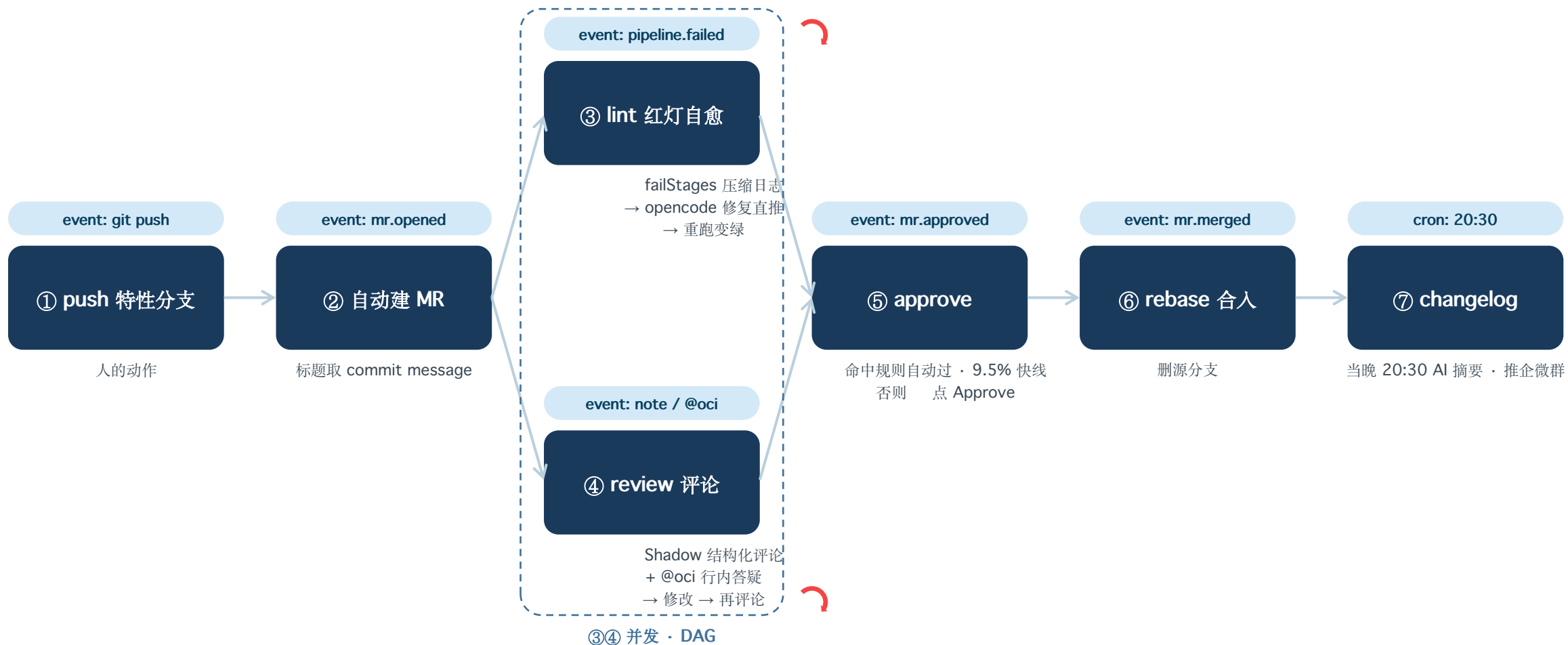
第二层

Agent in CI loop

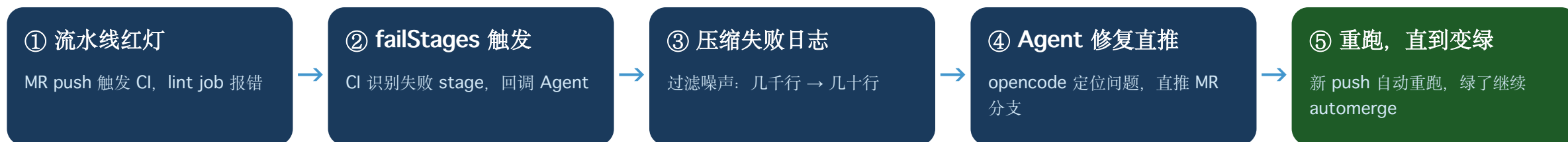
从麻烦中寻找价值。

一个 MR 的一生：七节点全自动走查

- 事件驱动：每个节点由 git 平台事件触发 (event 标签)
- ③ lint 与 ④ review 并发 (DAG)
- = Agent 自转 loop



CI 故障自愈：红灯不等人，自己修到变绿



没变绿就再修一轮: failStages 再触发, Agent 再修再推, 直到通过

几千行 → 几十行

日志先压缩再给 Agent, 省 token 也提高命中率

直推 MR 分支

修复不等人拷贝, 直接 push 回 MR 触发重跑

0 人盯屏

从红到绿全程自动, 人只看 review 评论

红灯不再是「叫人」的信号, 而是「叫 Agent」的信号。

failStages 配置: 把『红灯』接回 Agent

```
- name: ci_lint_autofix
  stages:
    - name: 各种验证
      jobs:
        gotest: go test
        pytest: pytest
        rufflint: ruff check
        pylint: pylint
        eslint: npm --prefix app/ugcpreview run lint
        golangci-lint: ./custom-gcl run
        doclint: ./optools lint doc
        featuredoclint: ./optools lint feature-doc -b ... -s ...
        protolint: ./optools proto lint
  failStages:
    - name: auto-fix-lint
      if: '["${ORANGE_BUILD_FAILED_STAGE_NAME}" = "各种验证"]'
      script: |
```

三组 linter, 谁负责哪一段

开源 / 官方 linter

gotes
t
.
pytest · rufflint · pylint · eslint

迭代 linter

golangci-lint (custom-gcl)

自研 linter

doclin
t · featuredoclint · protolint

Changes 24 Comments Checks 5 Commits 10 Conversations 16

Version 12, pushed at 2026-08-10 18:15:03

optools committed 2026-08-10 18:14:54

fix(): 修复 dispatch.py ISC004 隐式字符串拼接 lint 错误 ...

Version 11, pushed at 2026-08-10 18:13:39

radaren committed 2026-08-10 18:13:06

fix: 强化 : dispatch 状态回写与 delivery 真实回读 ...

Version 10, pushed at 2026-08-10 18:09:12

radaren authored 2026-08-10 18:08:03 radaren committed 2026-08-10 18:09:04

fix: 恢复 issue 124 验收证据并补齐 hooks/pre-commit ...

Version 9, pushed at 2026-08-10 18:08:30 Forced Push

optools authored 2026-08-10 18:04:52 optools committed 2026-08-10 18:08:16

fix: 修复 Lint 阶段 cyclop/funlen/intrange/mnd 告警 ...

Version 6, pushed at 2026-08-10 17:58:26 Forced Push

radaren authored 2026-08-10 17:57:14 radaren committed 2026-08-10 17:57:56

完成 issue 125 CoreDump 风险闭环 ...

auto-approve 快线: 153 个 MR (9.5%)

auto-approve (153 个)

8.5 min

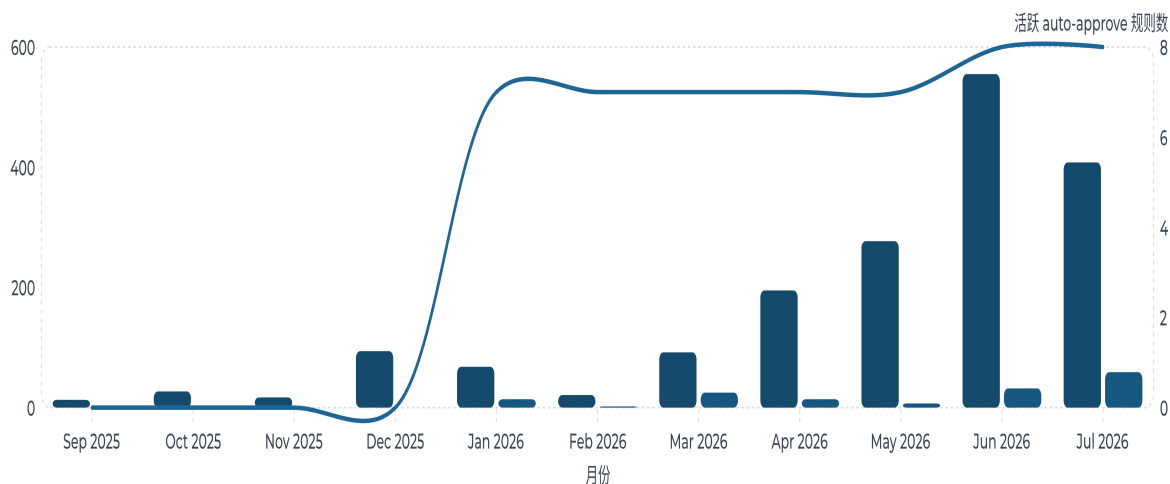
平均合入耗时
中位 2.2 分钟

394

lint 修复提交

Agent 自动完成

■ 已合入 MR 数 ■ auto-approve MR 数 — 活跃 auto-approve 规则数



注: $9.5\% = 153 / 1,616$ (已合入 MR 总数); 快线平均合入 8.5 分钟

工蜂 1/28 14:30:30

orangeCI 新建合并请求 (代码评审)

标题: refactor: 清理 CodeBuddy 框架废弃组件并优化目录结构 (!242)

项目:

描述: refactor: 清理 CodeBuddy 框架废弃组件并优化目录结构

- 移除废弃的 golangci-lint 相关 agent 文件
- 删除翻译相关的旧 agent 实现
- 将 doc-translator skill 从 skills/mr 重组到 skills 根目录
- 新增变更记录文件用于版本管理

变更记录: .changeset/refactor-codebuddy-agent-cleanup.md

Co-Authored-By:

评审人: optools

自动合线原因 1/28 14:31:43

【Auto Approve 放通通知】

阶段二 - 基于文件内容规则放通

Commit Message: refactor: 清理 CodeBuddy 框架废弃组件并优化目录结构

放通规则统计:

- Changeset 文件放通 (changeset_only) : 1 个文件
- 仅删除 (only_deletion_per_file) : 4 个文件
- 纯重命名 (pure_rename) : 1 个文件

工蜂 1/28 14:32:22

orangeCI 已完成代码合入

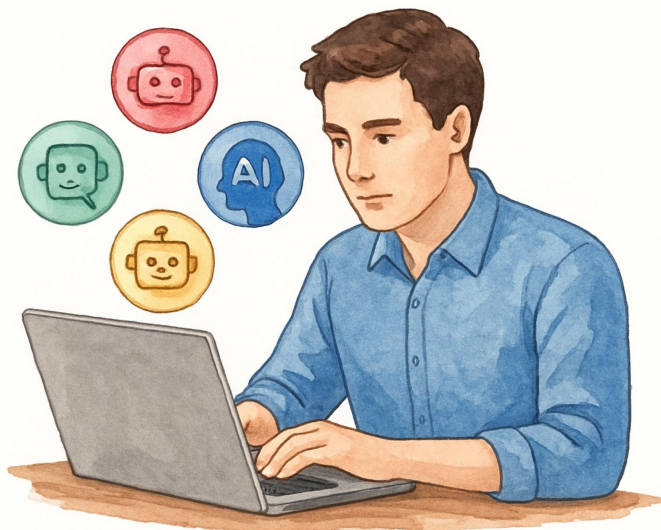
标题: refactor: 清理 CodeBuddy 框架废弃组件并优化目录结构 (!242)

17 / 51

Multi-Agent Review Loop: MR 评论反馈闭环

编写循环 (Writing Loops)

“你得主动询问并修复，这段代码是否冗长或有优化空间？”



MR 评论反馈闭环



oci-ask: 行内提问

业务仓配置驱动，按 diff hunk 生成行内 @oci 提问。严格校验每条提问有真实回复，防止无效问题堆积。



note-event: 线程作答

MR 评论里 @oci 由 OpenCode 作答，回到原行内线程。保持上下文连贯，不另起讨论。



--reject: 反馈建 Issue

对 Agent 回答不满意，@orange-ci (cnb) --reject 自动建反馈 Issue，关联 MR、评论锚点、文件行号和构建日志。

Multi-Agent Review Loop: 评论现场实录

Multi Agents Show · 上一页的三条机制，在真实 MR 评论线程里长这样

ng.txt'), `python3 tools, /verify\_issue89\_evidence.py <证据目录>` PASS; `./optools lint` 通过 (warnings=1, 仅既有 decisions.md 大小提示); mock=false; D EV 临时开关/零值注入/进程均已恢复。 |

Comments on line 239

optools
2026-08-10 10:27:48

@oci .codebuddy/skills/featur /reference/delivery-status.md:239 请将本次 Markdown 改动总结为严格30-50字的一句话，包含目的、影响和一个主要风险；只输出该句，不要标题、列表、表格、代码块或展开分析。

optools
2026-08-10 10:28:35

构建日志: <https://orange-ci.woa.com/build/log/oci-89>

校正: 89证据提交号使交付表引用真实commit, 影响复核; 主要风险是提交号写错会误导证据定位。

如果这条回答不准确, 请在当前线程 Reply @orange-ci --reject <具体问题>, 系统会自动创建反馈 Issue。

1 oci-ask: @oci 行内提问

按 diff hunk 生成行内提问，锚定在 239 行，带上文件与上下文。

2 note-event: 线程作答

OpenCode 回到原行内线程作答，附构建日志，上下文不另起讨论。

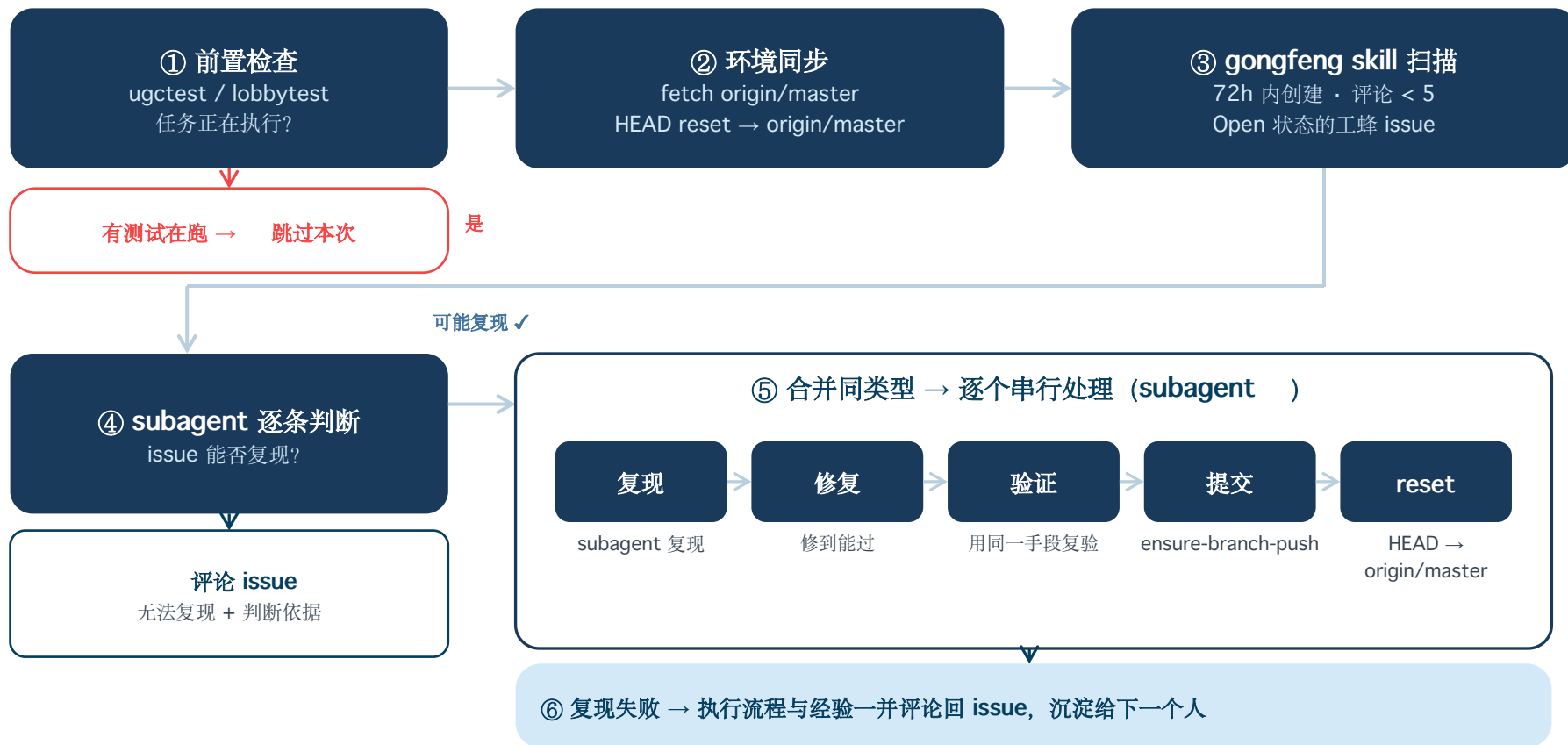
3 --reject: 反馈入口

回答不满意？在当前线程 Reply --reject，自动建反馈 Issue。

19 / 51

定时任务①：工蜂 issue 自动解决

cron: 每天 06:00-12:00 / 14:00-20:00 每小时触发 · 基于 git 平台 MCP · 全程无人值守



依赖

skills

- gongfeng
- ensure-branch-push

MCP

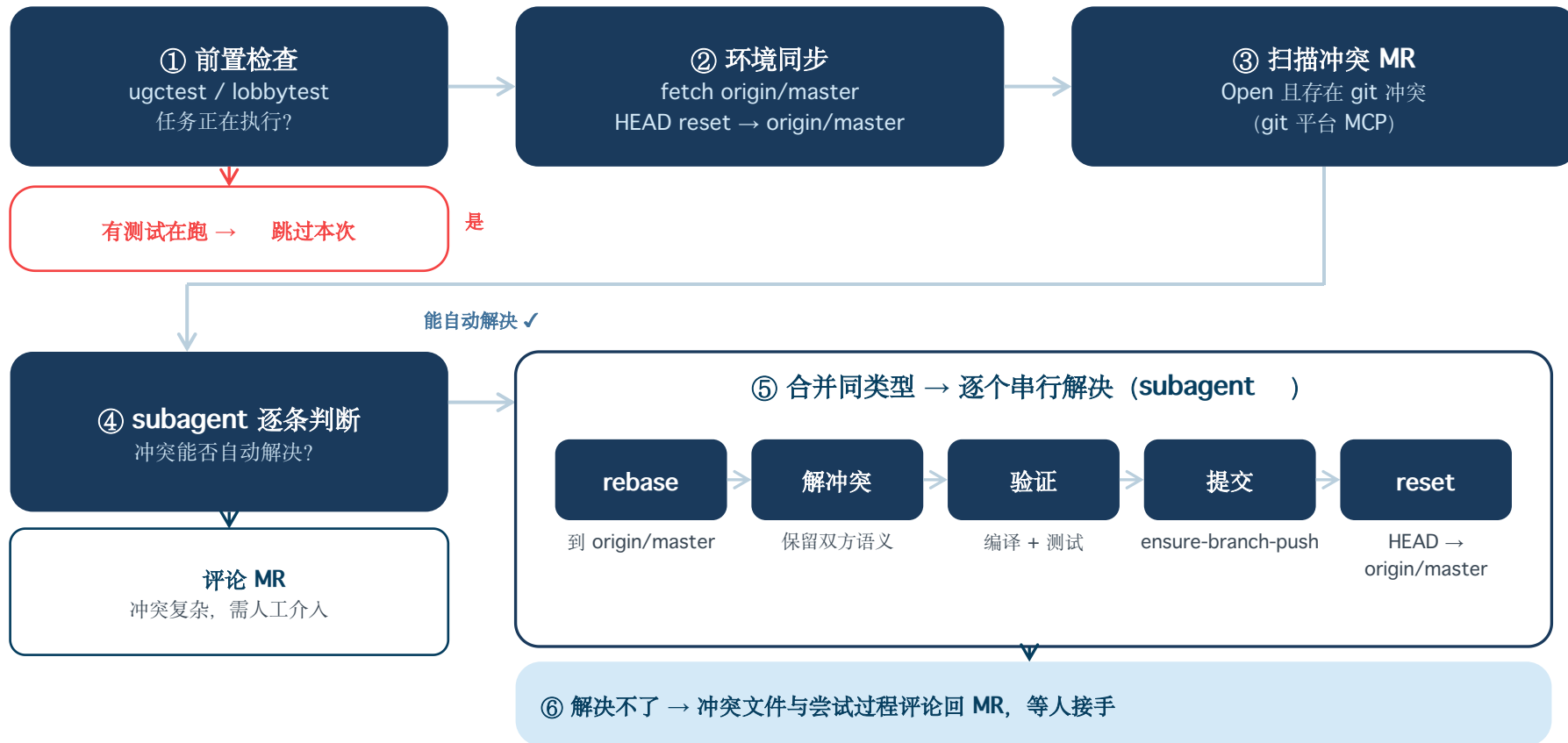
- git 平台: issue
- 评论 / 状态扫描

人休息, loop 值班 —— 能复现的自己修, 修不好也把现场留在评论里。

20 / 51

定时任务②: MR git 冲突自动解决

cron: 每天 06:00-12:00 / 14:00-20:00 每小时触发 · 基于 git 平台 MCP · 全程无人值守



依赖

skills

- ensure-branch-push

MCP

- git 平台: MR
- 冲突检测 / 评论

冲突先让 loop 试一刀, 解不开就把尝试过程留在 MR 里等人。

21 / 51

第二层 · 注意力答卷: CI loop

分钟级循环 · 人只在 REVIEW 点出现



人只看什么

- auto-approve 无法覆盖的复杂架构变更
- @orange-ci--reject 产生的反馈 Issue (Agent 判断失误的复盘)
- 发现 loop 本身的问题: 误放行、漏拦截、指标异常、Agent 反复自转但无法收敛

Agent 自转什么

- push 即建 MR, 标题取 commit message
- lint 自愈: 压缩日志 → 修复 → 直推分支
- review 结构化评论 + @oci 行内答疑
- 合入后 changelog 生成, 推团队频道

预告: 没有预算门, 一天几万行也能合进去。现场在第二章。



分享整体结构

开场 | Loop 的本质 = 注意力分配

第一章 | Loop Engineering

① 本地 hook loop

秒级 · 人不出现

② CI loop

分钟级 · review 点出现

③ work loop

小时/天级 · 守边界标准

④ graph loop

组织级 · 连成一张图

第二章 | Engineering Loop

故事一

200k 装不下

故事二

递归雪崩

故事三

4.4 万行

尾声 | RSI: 优化 loop 的 loop × 生活

故事旁色点 = 该坑被固化回哪一层

第三层

work loop: 事前-事中-事后

信任结构而不是概率: “/goal 你帮我 deep deep review 这个 MR”

本层核心问题

一天能做出多少高质量判断?

2

人的注意力只在 2 个高价值节点出现 —— 事前定边界，事后做判断。

■ 两层 Harness: Agent-CLI 侧 + 项目侧

同一套 harness, 两个挂载面

Agent-CLI 侧 Harness

托住事中, 让 `/workflow:work` 更敢 Yolo

挂载面	Codebuddy · Codex · dsh · OpenCode 等 CLI
托住什么	事中执行: 缓存命中率 (prompt 前缀稳定)、模型兼容性 (subagent / mcp / skill frontmatter 各种实现)
回灌到哪	hooks、linter、skills、workflow
目标	模型适配度

项目侧 Harness

拆工作与激发更强的 Agentic

挂载面	项目如何组织工作: <code>max_turns</code> 预算 + 循环检测、上下文压缩重注入、MCP / Skills 跨 CLI 通用
托住什么	输入和沉淀: 持续产生清晰输入, 并把执行结果沉淀回项目
回灌到哪	项目的文档、规则与知识库
目标	成本、供应稳定性、合规

白天做人该做的 clarify、review、test; 夜里让 Harness 接着跑, 并把结果继续回灌到 hooks、linter、skills 和 workflow 里。

24 / 51

work loop: 把人的注意力切成三种模式

小时/天级 · WORK



核心纪律: 不要一直看着 Agent 执行, Review 放在 MR, Test 放在 Hook/Cron, 执行交给 Agent。

人的注意力只在两个高价值节点出现: 事前定边界, 事后做判断。

MR 评论 → Skill 解决 → Hook / Linter / Auto-approve

Cron 失败用例 → 错误日志 → 自动修复

25 / 51

事前 Clarify: 先问清楚, 再动手

事前 · CLARIFY

核心纪律: 设计没被批准前, 不写一行代码。

探索 | 先自己找答案

输入: 模糊需求

先翻文件、文档和近期提交; 能在代码库里查到答案的, 不拿去问人。

输出: 项目上下文

追问 | 一次只问一个

输入: 项目上下文

沿设计树逐分支追问, 每个问题附推荐答案; 给出 2-3 个方案, 带权衡和推荐。

输出: 共识与方案

确认 | 批准后才动手

输入: 候选方案

分段呈现设计、逐段确认; 落成设计文档, 再自审一遍规格。

输出: design 文档

借自 Superpowers · Brainstorm: 批准前禁止任何实现

借自 Mattpott · Grill-me: 能查代码的, 不问人

事前多问一轮, 事后少返工一天——澄清消耗的 token, 比做错方向便宜得多。

26 / 51

■ 事前：把非结构化需求变成可执行方案

事前

需求结构化

需求结构化：从一句话到 YAML

为什么目前许愿Vibe的产物还不能跳过技术评审？

```
# proposal.yaml
```

boundary: 匹配超时归属与影响面

constraints: 热更新还是重启生效

acceptance: 「做完了」的测试清单

risks: 在匹配中的玩家

1

边界

匹配超时是客户端超时还是服务端超时？
影响哪些服务？

2

约束

超时时间从哪读？配置热更新还是重启生效？

3

验收标准

怎么算“做完了”？需要哪些测试？

4

风险

会不会影响正在匹配中的玩家？

27 / 51

事中：查杀分离，提高测试有效性

事中

Push (覆盖率驱动)

把覆盖率当 KPI，每个函数补单测，用例越堆越多。改一行代码挂一片测试，到最后没人敢删。

结果：测试是负担，不是资产。

Pull (风险驱动)

先问「这个场景失效的影响是什么」，再决定要不要写用例。tester skill 固化「怎么测」，「该测什么」留给人和风险判断。

结果：668 条用例，每条都有存在理由。

案例 1 | ML 训练集 / 验证集分离

拿训练集验证训练集，指标很好看、上线就翻车 —— 评估者必须独立于被评估的数据。

案例 2 | 业务安全的查杀分离

「查」和「杀」是两条独立链路：同一条链路既判断又处置，判错了没人拦 —— 分离，就是不让错误在同一系统内自我掩盖。

案例 3 | Agent 自写自测

写代码 + 写测试 + 跑测试共享同一上下文 → 认知同源、系统性盲点。
解法：上下文隔离 / 跨模型验证 / expected 前置声明。

论文证据：查杀分离有据，同源自测不可靠

31-49% → 6%

单模型「错法一致」的错误集群占比，跨模型集合后骤降 —— 异构才是有效隔离 (arXiv 2026-03 · 集合语义熵研究)

人写的测试，成功率更高

人写测试给 LLM (GPT-4 / Llama 3) 生成成功率一致更高 —— 前提是测试来自人 (ASE 2024)

3 个开源项目

属性测试在 NumPy / HuggingFace / AWS Powertools 发现真实 bug —— 属性比用例更难作弊 (arXiv 2025-10 · 属性测试 PBT 研究)

「删测试」

Kent Beck: Agent 会删掉测试让代码「通过」—— 测试的所有权必须留给人

28 / 51

单人并发度从 5 到 20 的升级

事中

plan 不是文档，而是状态机；lint 不是报告，而是 Agent 下一轮行动的输入。

01 | 触发

结构化 plan 落成，任务拆成节点进入 DAG，状态机开始调度执行。

02 | 反馈

节点状态与门禁（lint / test）结果实时回灌状态机：失败重试或降级，不让错误静默通过。

03 | 收口

DoD 校验通过才进入 MR；异常挂起等人决策——收口权始终在手里。

WORKSPACES

多 checkout + preview 分配 (含目录)

五条生产/数据线是最低覆盖面，不是并发上限；继续填满可用 hk/svr checkout，并标注冲突与共享 DEV 协调点。

目录	ISSUE / 本地	施工 TMUX	间商 TMUX	状态	HEROKINGDOM MR	SVR MR
/data/home/radaren/hk0	— 空闲暂无真实 gate 的独立 issue	—	—	● 空闲暂无真实 gate 的独立 issue	—	—
/data/home/radaren/hk6 fix/oci-ask-release-no-gcc	#130·预分配依赖 #15 已满足软冲突：与 hk8 当前 mirror 改动后续 rebase；共享 DEV 错窗	—	—	● 待施工	无	—
/data/home/radaren/hk8	— delivery-status 唯一写入	—	—	● delivery-status 唯一写入	—	—
/data/home/radaren/hk9 sync/B814	#219·预分配今日 DDL 依赖 #217/#218 已满足软冲突：共享 DEV 与 #97/#118/#200 错窗；主 hk9，配对 svr 按需	—	—	● to verify	无	—
/data/home/radaren/hkradartcaplus fix/cicache-go-test-build	— 暂缓硬冲突：preview 施工固定 hk8；当前 hk8 与 hkradartcaplus 均非 clean，不能占用	—	—	● 暂缓硬冲突：preview 施工固定 hk8；当前 hk8 与 hkradartcaplus 均非 clean，不能占用	无	—
/data/home/radaren/hktest3 fix/oci-ask-latest-overwrite	— 空闲暂无真实 gate 的独立 issue	—	—	● 空闲暂无真实 gate 的独立 issue	无	—
/data/home/radaren/svr	— 空闲#97 已移至 svr3；可作 #219 配对仓	—	—	● 空闲#97 已移至 svr3；可作 #219 配对仓	—	—
/data/home/radaren/svr2 sync/B812-worldid-svr2	#118·预分配依赖 #104 已满足软冲突：验收工具与 #117/#200 共享 DEV 错窗；0 生产 LOC	—	—	● 待施工	—	无
/data/home/radaren/svr3 feature/ _#222	#97·预分配依赖 #58/#96 已满足软冲突：同服务且共享 DEV 错窗；先预检 AreaNum>=2	#97	#97 · v01-111158	● to verify	—	无
/data/home/radaren/svr4 fix/issue-svr-23-20260812103023-1138157	#200·预分配依赖 #98 已满足软冲突：共享 DEV 与 #97 错窗	—	—	● 待施工	—	无
/data/home/radaren/svr5 feature, _#288	#117·预分配依赖 #104/#116 已满足软冲突：与 #97/#118 错窗；按回滚后合同量设计	—	—	● to verify	—	无

并行拓补 (互不共用 checkout)

svr
空闲#97 已移至 svr3；可作 #219 配对仓

hk9
#219 to verify

hkradartcaplus
暂缓硬冲突：preview 施工固定 hk8；当前 hk8 与 hkradartcaplus 均非 clean，不能占用

svr5
#117 to verify

svr4
#200 预分配依赖 #98 已满足软冲突：共享 DEV 与 #97 错窗

hk6
#130 预分配依赖 #15 已满足软冲突：与 hk8 当前 mirror 改动后续 rebase；共享 DEV 错窗

hk0
空闲暂无真实 gate 的独立 issue

hktest3
空闲暂无真实 gate 的独立 issue

svr2
#118 预分配依赖 #104 已满足软冲突：验收工具与 #117/#200 共享 DEV 错窗；0 生产 LOC

svr3
#97 预分配依赖 #58/#96 已满足软冲突：同服务且共享 DEV 错窗；先预检 AreaNum>=2

29 / 51

事中：Agent 并发的两种隔离模式

事中

"Concurrency is about dealing with lots of things at once. Parallelism is about doing lots of things at once." — Rob Pike

并发不是为了同时做更多事 而是为了选择冲突暴露的位置与时机

模式一：物理隔离 | 工作区互不共享 —— 冲突合线时才暴露

长期多目录

hk / hktest / hk2 / hk3

适合 多条需求线、长期服务、不同本地环境并存

收益 路径、worldID、构建产物关系稳定

代价 fetch / 切分支 / 清理都要逐目录维护

短生命周期 Worktree

multi-test-ugc 默认 3 个

适合 自动化测试、隔离验证、短生命周期分支

收益 创建轻、fetch 方便、适合批量并行

代价 worldID 显式隔离 (199-201)；残留进程跑完必清

模式二：逻辑隔离 + 收口治理 —— 冲突在过程中暴露

同目录多 Agent

共享同一代码现场

适合 proto / 业务 / 测试 / 文档强耦合需求

收益 冲突在过程中暴露，像 hook 一样持续校正

代价 需要任务边界：task claim / 锁 / 状态文件

收口层 /conflict

+ 流水线 + 定时任务

适合 分支合线、rebase、生成文件交错修改

收益 检测 → rebase → 逐 commit 分析 → 语义合并 → 校验

代价 只管最后收口，不替代前置协作

怎么选：弱耦合放多目录 · 测试验证用 worktree · 强耦合留同目录多 Agent · 合线一律走 /conflict 收口 —— 隔离性最好的多目录，不一定效率最高

worktree 是工具，多目录是现场，/conflict 是收口机制，流水线是窗口保洁 —— 目标不是开更多并发区，而是每个窗口都能被稳定回收。

30 / 51

■ 事后: AI 写了这么多代码, 人工如何 Review

事后

4 个模型 × 10 路并行, 同一句 deep review

@Hy3 × 10 路

“你帮我 deep deep review 这个 MR”

@GLM 5.3 × 10 路

“你帮我 deep deep review 这个 MR”

@DeepSeek V4 Pro × 10 路

“你帮我 deep deep review 这个 MR”

@Kimi K3 × 10 路

“你帮我 deep deep review 这个 MR”

4 个并行 Review Agent → 人工处理 → 回灌记忆

01 代码缺陷

提炼新 Linter 规则 / Hook

02 测试不足

强化 Tester Skill 约束

03 Harness 缺陷

更新 Skill Constraints

04 设计讨论

优化 Clarify 提问模板



人工处理: 合并四路结论, 判断对错、拍板取舍



调用 solve skill: 把确认过的问题提炼成可复用规则



回灌 skills / rules 记忆

必须落 DB 持久化, 跨会话、跨人生效, 不只留在当次对话

Review 的终点不是评论, 而是写回记忆库、下次自动生效的规则。

31 / 51

事后案例：一条 lint 规则的诞生

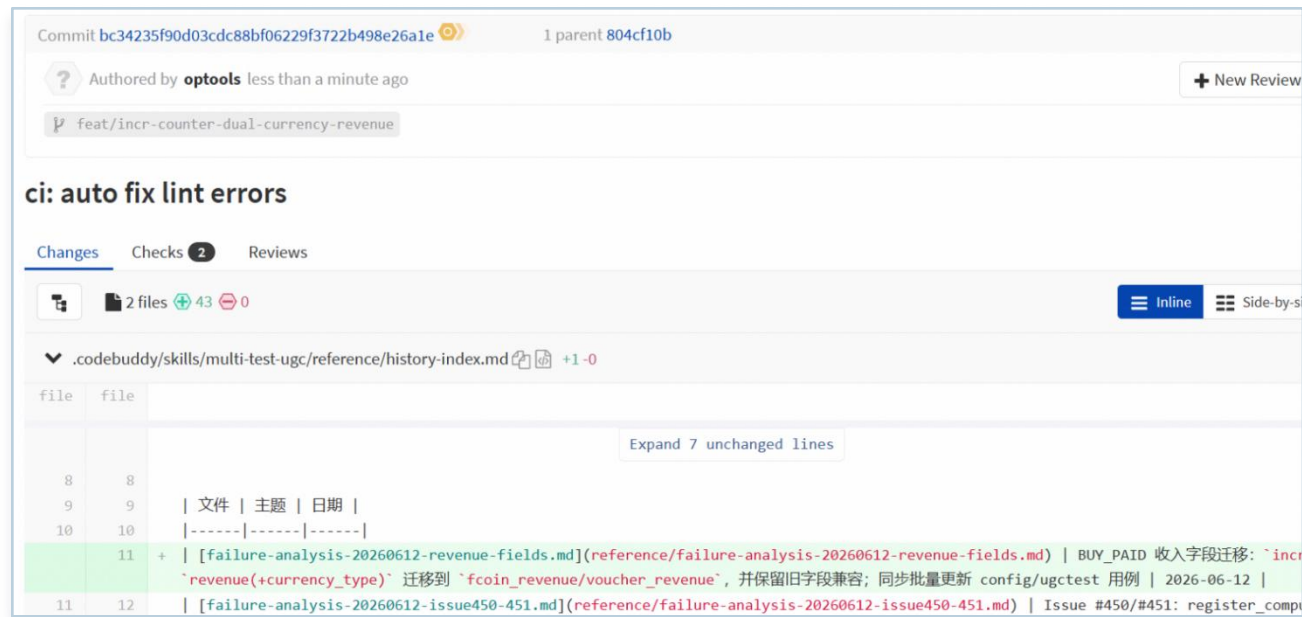
这就是上页「01 代码缺陷 → 提炼为新的 Linter 规则」的真实案例

企微群 · @所有人

加了一个 lint：单个 git 的 go lang 代码改动大于 50 行，需补充 markdown 文档说明变更——为了缓解 review 的认知负载。

一次 review 中的认知负载吐槽，当天沉淀为团队 lint 规则并写进 CI。

之后每个 MR 自动执行，不再消耗人的 review 注意力。



MR 现场：ci · auto fix lint errors

review 吐槽

lint 规则

CI 自动执行

怎么知道这套 loop 在变好

work loop 三阶段：人的注意力放在哪

事前 · Plan

人：做什么 —— 边界、验收、计划
AI 引导，人决策

事中 · Execute

人：不参与 —— AI 独立执行
Harness 托底

事后 · Review

人：做得对不对 —— Review、Test、沉淀
AI 辅助回灌

三个指标：怎么知道 loop 在变好

指标一 · MR 评论数

↓ 变好



总量往下走，说明前面两层拦住了更多问题，人不用再重复说。

指标二 · 重复评论占比

↓ 变好



同一类问题被反复指出，说明它该变成 hook 或 linter —— 回灌的报警器。

指标三 · 事前澄清耗时

↓ 变好



clarify 模板吸收历史 MR 的「设计讨论」类评论后，AI 提问越来越精准，事前要补的边界越来越少。

不看「写了多少代码」——看同类问题有没有变少，人的介入有没有变少、变准。

33 / 51

第三层·注意力答卷: work loop

小时/天级循环·人只守边界和标准

人只看什么

- 事前做选择题
- 事后确认 review 评论

两个高价值节点: 事前边界、事后判断

Agent 自转什么

- 事前拷问
- 事中执行
- 事后 review、修复、沉淀

其余全部交给 Harness 与 Hook 托底

从"同时关注三件事"到"每次只关注一件"

三种模式详见本层开篇 (work loop) —— 事前决策模式·事中释放模式·事后审查模式

Attention 是稀缺资源——人一天能做出高质量判断的时间有限。三阶段拆分的本质,是把有限的 Attention 集中到最高价值的节点:事前的边界决策和事后的质量判断。中间的执行交给 AI,不消耗人的注意力预算。

第四层: Graph Engineering

分享整体结构

开场 | Loop 的本质 = 注意力分配

第一章 | Loop Engineering

① 本地 hook loop
秒级 · 人不出现

② CI loop
分钟级 · review 点出现

③ work loop
小时/天级 · 守边界标准

④ graph loop
组织级 · 连成一张图

第二章 | Engineering Loop

故事一
200k 装不下

故事二
递归雪崩

故事三
4.4 万行

尾声 | RSI: 优化 loop 的 loop × 生活

故事旁色点 = 该坑被固化回哪一层

前三层同一轴: 反馈半径——让一个 Agent 把一件事做对
第四层换轴: 组织半径——把多人的 loop 连成一张图

前三层做了什么

- 第一层: 把 agent 织进写代码的循环 (秒级, lint 自愈)
- 第二层: 把 agent 织进代码合入的循环 (分钟级, CI 自愈)
- 第三层: 把 agent 织进任务交付的循环 (小时/天级, DoD 校验)
- 共同点: 代码有 lint 和 CI, 代码的质量循环已经建好

第四层做什么

“文档 / Spec 和实现永远对不上”
不是 design/apply 知识库, 而是 iteration 知识库
不是 design/apply skill, 而是 iteration skill
不是 design/apply workflow, 而是 iteration workflow

传统 SDD 框架最大的问题: 归档 (Archived) 的知识库, 再也没人看——不换轴、只堆文档, 就会死。

Graph Engineering: 把多个人的 loop 连成一张图

第四层 · 从 Loop Engineering 到 Graph Engineering——一个 loop 解决一个问题；连起来的 loop 网络才是团队产能

Graph loop 的复制：从个人练习到团队产能

个人 loop

一个 loop 解决一类结构问题，问题结构本身难以复制



复制 loop 本身

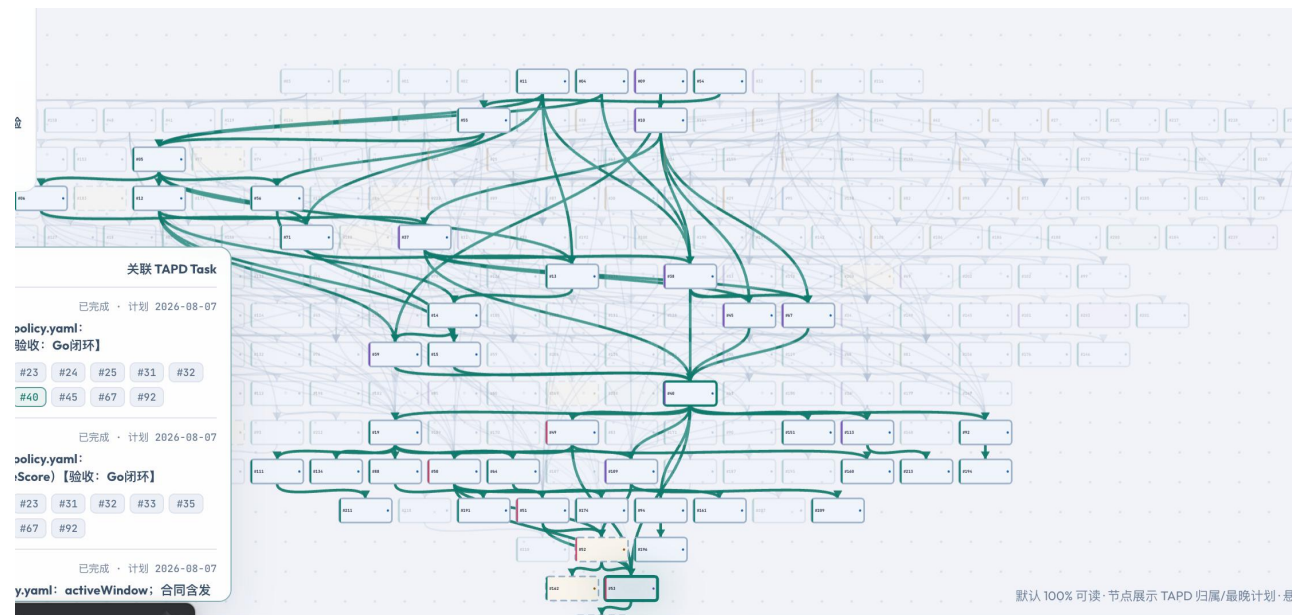
还是解决同一个问题——更像 exercise（练习）



团队级 loop: graph 化编排

多人的 loop 连成一张图：划清文件所有权、隔离 worktree、统一 review 口径——减少冲突，组织效率才出得来。

从个人 **exercise** 到团队 **productivity**：练的是个人手感，连起来才是组织能力。

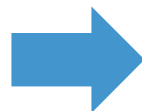


AI 时代，复制 loop 的成本越来越低——a little duplication is better than a little dependency

Graph Engineering: 从烟囱工作台到 DSH 插件化

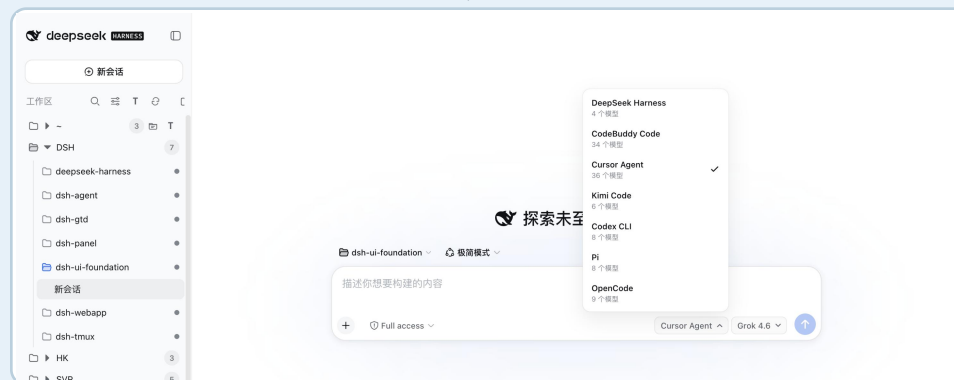
烟囱工作台 · 每人一套私有工具链

- hook、脚本、提示词散落在各人电脑，一人一套
- 好用与否全看个人手艺，质量参差、口径不一
- 经验锁在个人 loop 里，无法复用、人走即失



DSH 插件化 · 统一入口、插件共享

- 统一入口: DeepSeek Harness 一个工作台，人人同一起点
- 插件共享: 能力封装成插件，一次写好、全组复用
- 进仓库沉淀: 版本化、可评审，还能跨实现对比



“对比下你和 <https://github.com/shaokeyibb/dsh-plugin-product-subagents> 实现的差异”

37 / 51

Graph Engineering: 场景化 SDD

概念趋同

想办法让每个人都能 build loop

概念	feature-ugc (独立发明)	SDD 框架 (规范定义)
任务编排	<code>parallel_group: [1], [2,3], [4]...</code>	<code>parallel_groups: [[1], [2,3], [4]...]</code>
执行隔离	"每个 stage 使用 subagent fork"	<code>context: fork</code>
上下文传递	<code>~/sdd/ugcfeature/{name}.{id}.md</code>	<code>~/sdd/feature/\${name}/stage_\${id}.yaml</code>
stage prompt	<code>agents/step1-collect.md</code> 等 7 个文件	<code>stage_\${idx}.yaml</code> 由 plan 生成
主会话角色	"主会话作为编排器"	<code>workflow:work</code> 作为编排器
review 机制	<code>step7</code> 自检清单 (Explore agent)	<code>workflow:review-stage skill</code>

跑出来的结论比设计出来的更牢靠：`parallel_group` 比线性好、`subagent` 隔离比共享好、文件传递比内存可靠。实际路径 5 天 8 次提交、3 个业务域已投入生产——如果先套框架，可能还在走第一个 plan 的 stage。

Engineering Loop: 踩坑与反思

分享整体结构

开场 | Loop 的本质 = 注意力分配

第一章 | Loop Engineering

① 本地 hook loop
秒级 · 人不出现

② CI loop
分钟级 · review 点出现

③ work loop
小时/天级 · 守边界标准

④ graph loop
组织级 · 连成一张图

第二章 | Engineering Loop

当前章节

故事一
200k 装不下

故事二
递归雪崩

故事三
4.4 万行

尾声 | RSI: 优化 loop 的 loop × 生活

故事旁色点 = 该坑被固化回哪一层

Engineering loop 不讲抽象框架，只讲真实约束：三个失效现场，以及它们后来怎样被固化成 hook、CI、work loop 或 skill。

故事一 | 200k 装不下的 feature-sdd

巨型 skill 的 token 膨胀：拆成可按需加载的知识包，200K 上下文才装得下——回指第四层燃料：知识不可复制，就会撑爆上下文。

故事二 | subagent 递归 loop

两场 7000 层递归雪崩：Skill 自己调自己，没人喊停，734M token 烧在空转上——回指前两层：缺终止条件。

故事三 | 一天 4.4 万行代码

4.4 万行 diff 怎么滚出来的：八步腐化链，27 轮评审也没拦住——回指第三层：缺预算门。

■ 故事一 | 200k 装不下的 feature-sdd

很多模型标称 1M 上下文，但 200K 仍是一道硬阈值：超过它，模型兼容性变差，「智能区间」也被击穿。

随着业务迭代的 feature-SDD

270k

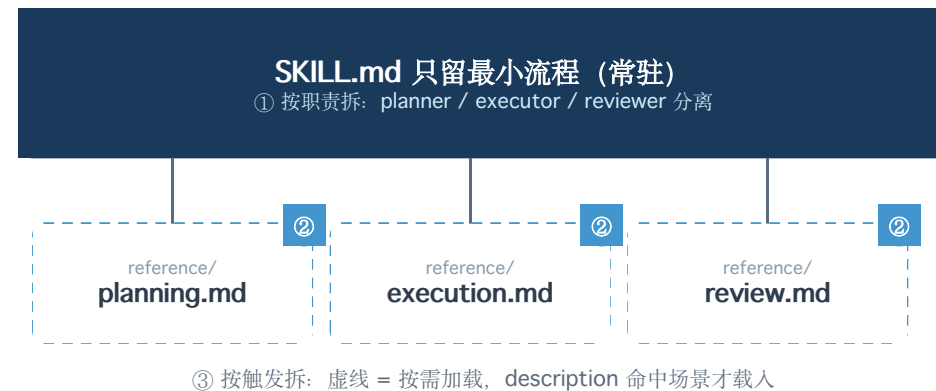
explore 一个任务的实测 Context 消耗

670k

完成同类任务的 Token 消耗；巨型 skill 让低成本模型无法承接。

拆分前
巨型 feature-SDD 单体
270k+ Context

三刀拆



01 按职责拆

planner / executor / reviewer 分离，避免一个 skill 同时负责规划、执行和验收。

02 按阶段拆

SKILL.md 只保留最小流程；细节进入 reference/planning.md、execution.md、review.md。

03 按触发拆

description 写清楚何时使用；命中当前场景才加载对应 reference，否则建议 Agent 去读路由 Skill。

新增 linter 盯四个指标：同类任务 Token、subagent fork 深度、reference 加载次数、SKILL.md 行数。

故事二 | 同一套 SDD: 正常那晚 vs 失控那晚

2026-05-26 · 事后用 3 路子 Agent 复盘 21,458 份 jsonl



后来怎么修

- P0 skill 正文加守卫: 已在子代理内, 禁止再调同名 Skill
- P0 清掉旧版本插件缓存, 让修复真正触达运行时
- P1 同名递归 $\text{depth} \leq 1$; 单会话子代理 >50 直接熔断

同一套 SDD, 正常那晚 8 分钟 24M token 收工; 失控那晚 7,000+ 层递归烧掉 734M——差别不在任务复杂度, 在有没有终止条件。

42 / 51

故事三 | 一天 4.4 万行：代码是怎么失控的

2025-12-30 · tbreport 重构分支 · 从「先生成一份报告」开始，32 个子代理把它滚成了 44,226 行的 diff

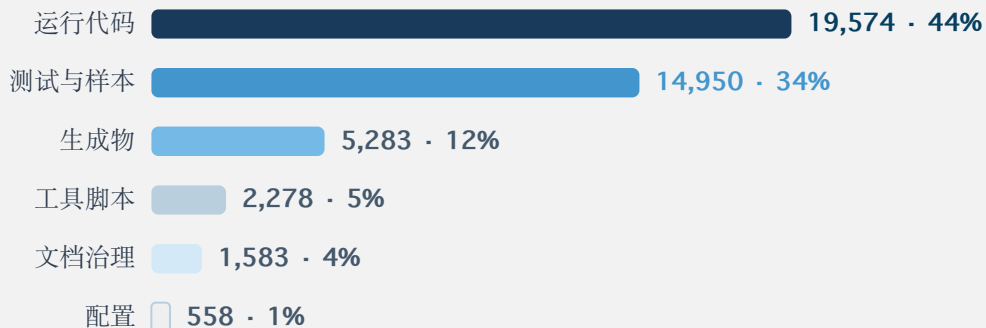
「Loop 没有跑偏，它精确执行了我给的错边界」

tbreport 管道架构

注：服务本身不提供 RPC



4.4 万行里都有什么



直接产出不到一半，其余都是一层层加上去的「加固层」。

结局：不合并

- 不合入、不搬 commit、也不在主干重做一遍
- 复核事实后，用一个小 patch 把修复落回主干
- 最难的一步，是承认「这次扩张本身就不该发生」

评审最后自己写下「合并会破坏主干」——判断没错，只是迟了 27 轮，这句话本该出现在第一天。

43 / 51

故事三 | 腐化链：八步滚成一个 4.4 万行的 MR

43 工作项 + Gate A-E → finish-all，每一小步都过了门禁；同一棵工作树、同一个分支，每一小步看起来都合理——连起来就是失控



MR !1785 的评审现场

221 个评审发起项

27 轮评审 runs

Hard Gate 每轮最多放 10 条

最后 5 轮 0/72 文件被覆盖

审查工具最后盖不住它自己帮忙吹大的 MR——规模本身就把流程打败了。

缺的那一环不是更聪明的审查，而是每轮开始前重新确认「最小交付边界」——先问该不该做，再问做得好不好。

故事三 | 回收成规则：给 loop 装刹车

代价是 4.4 万行废弃代码，换回六条写进流程的门禁

1 先写一页纸

刹 ①②

开工前写清「必须有什么、明确不做什么」。finding 默认不是 todo，批准了才算。

2 变更预算门

刹 ⑤

diff 超 3,000 行或 40 个文件，强制停下来——先比较替代方案，再决定要不要继续。

3 工具与业务分家

刹 ⑥

修审查器的代码走独立分支、独立 MR，不和业务变更混在一起——运动员不兼裁判。

4 并行先划界

刹 ③

每个代理认领文件所有权，或用独立 worktree；reviewer 固定 tree hash，不看会变的代码。

5 先删再加

刹 ④

要加新机制，先把过时的 harness 删掉。防御层只减不增，代码才有被看完的可能。

6 如实记 BLOCKED

刹 ⑦⑧

缺证据就老老实实写 BLOCKED，不许用猜测把报告填满——承认卡住也是交付。

loop 不只要会转，还要会停——刹车不是拖慢效率，是让效率不白烧。

第二章收口 | 回头看：三次失控，同一道题

巨型 skill 撑爆上下文 · Skill 自递归 · 错边界无预算门——症状不同，病根相同

	故事一 200k 装不下	故事二 递归雪崩	故事三 4.4 万行
失控机制	巨型 skill 撑爆 200K 上下文	Skill 自递归，无终止条件	错边界被精确执行，没有预算门
缺的刹车	知识按需加载	递归深度守卫 + 熔断	每轮前重确认最小交付边界
固化回哪层	④ graph loop · 知识分层	① hook② CI	③ work loop · 守边界

三次失控症状不同、病根相同——loop 会转不会停；刹车要提前装进流程，不是事后追认。

Cloud Agent 从 Prompt in Cloud 转向 Loop in Cloud

OpenClaw: tick 让 Agent 有生命

① tick 定时触发

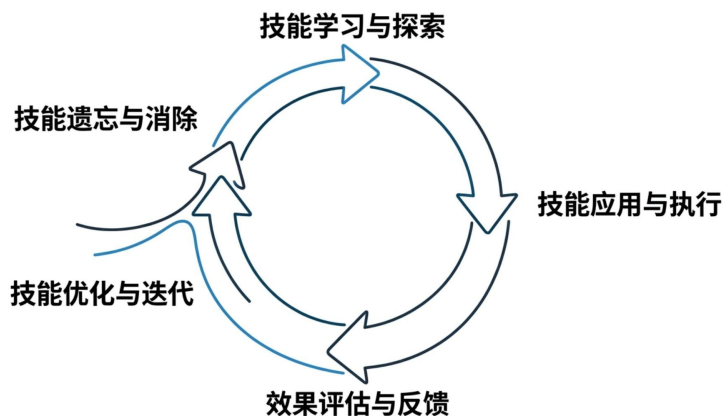
心跳 + cron 定时唤醒: Agent 按节奏自己醒来, 有事做事, 没事静默。

② 插件化记忆机制

记忆可插拔: markdown 记忆文件打底, 记忆插件 (如 MemOS) 按需替换。

③ Agent in the IM – Claude Tag?

Agent 住进 IM: WhatsApp / Telegram 里 @ 一下就派活, 对话即入口。



Hermes: 机制化成长Skill

① 轮次触发 (Turn-based)

每 `_skill_nudge_interval = 10` 轮用户对话自动触发一次 skill review。

② 工具调用量触发 (Tool-iteration Threshold)

单轮内工具调用次数 ≥ 10 次, 判定任务复杂, 触发 skill 沉淀评估。

③ 打断/失败修复 (Interruption Fix)

review 进程重启时从断点恢复, 打断不再丢失累积进度。

④ 技能遗忘与淘汰

Memory 有效的前提是会遗忘: 低频、低效的 skill 会被降级乃至移除, 技能库不膨胀, 上下文才不过载。

Loop 是迭代, 不是重复

- 迭代(iteration): 螺旋式推进, 每次有不同的输入/状态, 方向可调整
- 循环(cycle): 周而复始回到原点, 无累积增益

47 / 51

下一站 | RSI: 优化 loop 的 loop

前面讲的 loop 都在跑任务；下一个阶段，是 loop 回过头来优化自己

RSI 递归自我改进: 优化 loop 的 loop

每晚定时任务，review 当天全部 agent session



找出重复踩的坑：哪条 rules 没生效、harness 哪里卡



直接改 rules 和 harness，第二天生效

今天的 session 是明天规则的**训练数据**——loop 跑的不只是代码，还有它自己的规则库。前面两起事故的复盘，就是这个 nightly review 的雏形。

回顾 p47 RSI 的本质：结构化提效

OpenClaw: tick 定时唤醒，Agent 按节奏自己醒来

Hermes: 轮次 / 工具量触发 skill review，会遗忘才不膨胀

共同点: 提效不靠人盯，靠结构化的触发机制

RSI: 把这套机制推到规则层

p47 是结构化提效的雏形；RSI 让 loop 每晚改自己的 rules 和 harness，第二天生效

前三层织进去，第四层连成图；下一站，让这张图改自己的规则。

■ 收尾 | 开场三问的答卷

开场三问 · 结尾回收——现在作答

① 什么时候 in / on / out the loop?
(Token ROI)

in | 事前定边界，事后做判断
on | review 点：MR 评论线程
out | hook / CI / cron 自转

—— 对应四层 loop 的人事分工：人只出现在判断点，
执行全部交给 loop。

② Loop 多花几倍 token，是否也有心流价值？

不看「写了多少代码」。

看同类问题有没有变少、人的介入有没有变少、
变准。

—— 三个指标：MR 评论数 · 重复评论占比 · 事前澄清
耗时（见第二层·质量答卷）。

③ 超级个体是否还有价值？

human in the loop,
human build the loop.

人从执行者，变成规则与边界的作者。

—— 超级个体的价值，从「写得快」变成「把判断沉
淀成规则」。

Loop 也在生活里转

工程 hook 与生活 loop 的结构同构：触发条件 → 执行 → 反馈信号 → 修正

反馈半径：月/年级 · 定期观测

体检是健康的 loop

定期观测身体指标，提升身体素质——用数据而不是感觉管理健康。

触发：年度体检

反馈：指标报告 → 调整生活习惯

反馈半径：天级 · 分钟-小时

构建自己的 GTD 工作台

GTD 收件箱是一个天级的 CI loop：把上下文给到 Agent 一起讨论。

触发：每日收件箱

反馈：清空、分类、排到日程

反馈半径：周级 · 回灌

反思回灌

没有回灌的 loop 只是重复，有回灌的 loop 才是成长。

触发：每周复盘

反馈：经验写回规则与习惯

人被 loop 赶着走，是注意力工程的失败；loop 应该让人保持积极性，而不是消耗人。

QCon

全球软件开发大会

上海
站

大模型正在重新定义软件

Large Language Model Is Redefining The Software

2026 年 10 月 22-24 日 / 上海建工浦江皇冠假日酒店



参会
咨询

**专题：从 App 到 Agent：字节跳动
Supabase 的 Agent-Native 规模化实践**

专题演讲嘉宾：张国强

字节跳动 / 资深技术专家



**专题：复杂业务 Agent 的持续进化：
快手电商导购的 Harness Loop 实践**

专题演讲嘉宾：包磊

快手 / 资深技术专家



**专题：从 Harness 到 Loop：阿福 Agent
小队如何处理持续涌入的线上 Badcase**

专题演讲嘉宾：肖汉松

蚂蚁集团 / 阿福 Harness 架构组负责人



**专题：AI Coding 在大型客户端工程中的
落地实践——从通才模型到专才 Agent**

专题演讲嘉宾：杨凯

小红书 / 客户端基础技术负责人



**专题：AI 时代的一人公司：从 Idea 到
Growth 的完整 workflow**

专题演讲嘉宾：casa

独立开发



**专题：Agent 驱动的全平台 GUI 持续
验证实践**

专题演讲嘉宾：王政达

微软 / SENIOR SOFTWARE ENGINEER



THANKS

行百里者半九十

Living in iteration rather than completion



扫码交流 · Loop Engineering 实践



人机 279
湖南 长沙



扫二维码，添加我为朋友。